

Control flow: retries, branching, looping

Retries, the six Run if conditions, If/else branching, For each loops, disabled tasks and repair runs.

Control flow is about the runs that **do not** go to plan. The happy path needs no configuration at all.

Retries

- ◆ **A retry policy re-runs a failed task** up to a set number of times before the task is considered failed.
- ◆ **You can set a minimum interval between retries**, so a struggling downstream system gets time to recover.
- ◆ **Errors are often transient**, and a restart resolves them. That is what retries are for.
- ◆ **Not all job configurations support task retries.**

ON THE EXAM

In continuous trigger mode, Databricks retries automatically with exponential backoff. You do not configure that.

A retry only helps if running the task twice is **safe**. Lakeflow Jobs does not make your tasks idempotent. Your task logic does.

The six Run if conditions

CONDITION	THE TASK RUNS WHEN
All succeeded	Every dependency ran and succeeded. The default.
At least one succeeded	At least one dependency succeeded
None failed	No dependency failed, and at least one ran
All done	Every dependency finished, whatever the outcome
At least one failed	At least one dependency failed
All failed	Every dependency failed

ON THE EXAM

All succeeded is the default. The last three exist so that failure handling and cleanup can be part of the same job.

Two different outcomes when the condition is unmet

- ◆ **All succeeded, At least one succeeded, None failed:** unmet means the task is marked **Upstream failed**.
- ◆ **At least one failed, All failed:** unmet means the task is marked **Excluded**.
- ◆ **Excluded tasks are skipped and treated as successful.**
- ◆ **If all of a task's dependencies are excluded, that task is excluded too,** whatever its own condition says.

ON THE EXAM

Upstream failed is a problem. Excluded is a cleanup task correctly deciding it has nothing to clean up.

A job's success or failure is determined by the outcome of its **leaf tasks** — the tasks with no downstream dependencies.

If/else tasks

- ◆ **Run if branches on the outcome of dependencies.** If/else branches on a value.
- ◆ It can test task values, job parameters, or dynamic values.
- ◆ Two outputs, true and false, and you attach different downstream tasks to each.
- ◆ It is a task in its own right, not a setting on another task.

ON THE EXAM

Row count is zero, so skip the rebuild. Environment is prod, so send the alert. Those are If/else, not Run if.

Run if or If/else?

RUN IF

Branch on outcome

- A property of a task, set on its dependencies
- Six fixed conditions about success and failure
- Answers: did the work before me succeed?
- Used for cleanup, fallback and failure handling

IF/ELSE

Branch on a value

- A task in its own right, with true and false outputs
- Tests task values, job parameters or dynamic values
- Answers: what does the data or configuration say?
- Used for skipping work, environment switches, thresholds

THE TIEBREAKER

If a scenario mentions succeeded or failed, that is Run if. If it mentions a number, a name or a flag, that is If/else.

For each tasks

- ◆ **Runs another task in a loop**, passing a different set of parameters to each iteration.
- ◆ **It requires a nested task**. The For each task alone does nothing.
- ◆ **The nested task is one of the standard task types**, such as a notebook.
- ◆ **Iterations can run concurrently**, which is the point: fourteen regions processed in parallel.

ON THE EXAM

For each exists because a DAG cannot contain a cycle. Looping had to become a task type rather than a graph shape.

Disabled tasks

- ◆ **Disabling skips a task at run time** without removing it from the job definition.
- ◆ **Disabled is neither success nor failure.** It is its own terminal state.
- ◆ **So a downstream task with All succeeded will not run**, because disabled does not satisfy the gate.
- ◆ **A downstream task with All done will run**, because disabled counts as done.

ON THE EXAM

An If/else or For each task whose provider task is disabled fails. Disabling upstream of control flow is not a safe way to pause work.

A repair re-runs only the **unsuccessful** tasks and their dependents. Successful tasks, and anything downstream of them, are not run again.

What repair does and does not guarantee

- ◆ It re-runs each unsuccessful task from the beginning. Not from where it stopped.
- ◆ Lakeflow Jobs does not make tasks idempotent.
- ◆ So if a task wrote part of its output before failing, re-running it can duplicate that data.
- ◆ Whether a repair is safe depends on the task's own logic. Merge is safe; append usually is not.

ON THE EXAM

This is the same idempotency question as module 10 and as retries, arriving for the third time. It is worth expecting.

Reading a failed run

- ◆ **The Runs tab shows a matrix view**, with each task's outcome across recent runs.
- ◆ **A task can be unsuccessful two ways**: it failed, or it was skipped because a dependency failed.
- ◆ **Hover a failed task for its metadata**, then open the task run for logs and the error.
- ◆ **Find the first failure, not the loudest one**. Downstream skips are consequences, not causes.

ON THE EXAM

The matrix view is designed for exactly this: seeing at a glance which task broke and whether it has broken before.

Cancellation

- ◆ Cancelling a task run propagates to downstream tasks.
- ◆ Tasks whose Run if handles failure still run, so a cleanup task fires on a cancellation.
- ◆ That is deliberate, and it is how you guarantee temporary resources get tidied up.
- ◆ A cancelled run can also be repaired, like a failed one.

ON THE EXAM

If a scenario needs something to happen 'whatever else occurs', All done covers success, failure and cancellation alike.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
A cleanup task must run whatever happened	Run if: All done
A task must run only when something failed	Run if: At least one failed
Skip expensive work when a count is zero	If/else on a task value
The same task for every region	For each with a nested task
Recover a failed run without redoing everything	Repair run
The repair duplicated data	The task is not idempotent. Repair re-runs from the start.
The job reported success but a task failed	Job status comes from leaf tasks; Excluded counts as success
A transient network error	A retry policy

ON THE EXAM

Eight lines. The sixth and seventh are the two facts most likely to surprise you on the day.

A job creates a temporary staging table, runs three transformation tasks against it, then drops it. The drop task currently depends on all three transformations with the default Run if condition.

When any transformation fails, the staging table is left behind. It has accumulated for months and is now large enough to matter.

What should the Run if condition on the drop task be?

- | | |
|--|--|
| ☐ A All succeeded, which is already correct | ☐ B All done |
| ☐ C At least one failed | ☐ D None failed |

◆ PAUSE AND DECIDE

What should the Run if condition on the drop task be?

A All succeeded

This is the current setting and the cause of the problem. When a transformation fails the condition is unmet, so the drop never runs.

C At least one failed

Inverts the problem. The table would now be dropped only when something failed, and left behind on every successful run.

B All done

Correct. The drop runs once all three dependencies have finished, regardless of outcome, so the staging table is removed either way.

D None failed

Still requires that nothing failed, so it behaves like the current setting in exactly the case that matters.

THE SIGNAL

Cleanup should not depend on the outcome of the work it is cleaning up after. All done is the condition that expresses that.

A job's third task appends new records to a fact table. One night it fails part way through, after writing roughly 60% of the batch. An engineer uses Repair run to recover, which succeeds.

The next morning, the fact table contains about 160% of the expected rows for that date.

What happened?

- ☐ A Repair re-ran the task from the beginning, and because it appends, the partial output was duplicated
- ☐ B Repair re-ran all tasks in the job, including the earlier successful ones
- ☐ C The retry policy fired at the same time as the repair, running the task twice
- ☐ D Delta failed to roll back the partial write, which is a known defect

◆ PAUSE AND DECIDE

What happened?

- A

 Repair re-ran from the beginning; the append duplicated
Correct. Repair re-runs each unsuccessful task from the start. Lakeflow Jobs does not make tasks idempotent, so a partial append is added to twice.
- B

 Repair re-ran everything
The opposite of what repair does. Successful tasks and their dependents are deliberately not re-run.
- C

 A retry fired alongside the repair
Invents a concurrent execution. The 160% figure matches one partial write plus one full write, not two full runs.
- D

 Delta failed to roll back
Delta writes are atomic per write. A task that committed several batches leaves those committed; that is correct behaviour, not a defect.

THE SIGNAL

60% written, then 100% written again, gives 160%. The arithmetic identifies the cause before you read the options.

A job ingests a daily file, then rebuilds a large gold aggregate that takes 40 minutes. On roughly a third of days the supplier sends no file, so there is nothing new, and the rebuild produces an identical result.

The team wants to skip the rebuild on those days. The ingestion task can easily compute how many rows it loaded.

How should this be built?

- ☐ A Set the rebuild task's Run if condition to At least one succeeded
- ☐ B Have the ingestion task set a task value with the row count, then use an If/else task to branch on it
- ☐ C Give the ingestion task a retry policy so it waits for the file to arrive
- ☐ D Disable the rebuild task on days when no file arrives

◆ PAUSE AND DECIDE

How should this be built?

A Run if: At least one succeeded

Run if branches on outcome, and the ingestion task succeeds either way. Loading zero rows is not a failure.

C A retry policy on ingestion

Retries handle failures. No file is not a failure, so there is nothing to retry, and retrying would only delay the run.

B A task value plus an If/else task

Correct. The decision depends on a value rather than an outcome, which is exactly what If/else is for.

D Disable the rebuild task

Disabling is a manual configuration change. Somebody would have to do it before each run, having already checked whether a file arrived.

THE SIGNAL

The ingestion task succeeds whether or not a file arrived. Nothing about the outcome distinguishes the two days — only the value does.

A job has a validation task that runs after three loaders, with Run if set to At least one failed, so it can log details and alert an on-call engineer. It is the last task in the graph.

One night a loader fails. The next morning nobody has been alerted, and the job's run history shows the run as Succeeded.

Why did the run report success?

- ☐ A The failed loader was automatically retried and eventually succeeded
- ☐ B Job status is determined by leaf tasks, and the validation task ran and succeeded as the only leaf
- ☐ C Run if conditions suppress the failure status of their dependencies
- ☐ D The run history shows the status of the last task only, which is a display limitation

◆ PAUSE AND DECIDE

Why did the run report success?

A The loader was retried and succeeded
Then the validation task would not have run at all, since its condition requires a failure. The scenario implies it did run.

C Run if suppresses dependency failure status
It does not change any dependency's status. It only decides whether this task runs.

B Job status comes from leaf tasks
Correct. Databricks determines run status from the outcome of leaf tasks. The validation task is the only leaf, it ran, and it succeeded.

D A display limitation
The status is computed, not merely displayed. Calling it a display bug misses a designed behaviour.

THE SIGNAL

The design is working. Put notifications on the task, not only on the job, or a handled failure becomes an invisible one.

Run if branches on **outcome**. If/else branches on **value**. And no mechanism here makes a task safe to run twice — **only your task logic does that.**